Problem Motivation:

I manage a monthly household budget that includes expenses related to shopping, dining out, gifts, education, health, and other everyday needs. While spending on both essential and discretionary items is enjoyable for me, it has become increasingly difficult to remain within my planned monthly budgets. I have a tendency to overspend, particularly on discretionary activities such as shopping, dining out, and social gatherings. As a result, overspending is common, and frequent item returns make it challenging to accurately distinguish between money that has been spent and money that has been refunded.

Budget tracking is further complicated because I manage expenses not only for myself, but also for my children and my husband, each of whom has a separate monthly spending allowance. In addition, expenses span multiple categories such as clothing, groceries, electronics, health products, dining, and education-related items, each with its own budget limit. Some expenses are incurred online, while others occur in physical retail locations. I have been tracking all of this information manually using a notebook, which is time-consuming, error-prone, and difficult to maintain over time.

These challenges motivate the need for an application that can record purchases and returns, track spending by family member and category, and provide clear visibility into overall monthly budget usage.

Project Title: Smart Budget Database System

Project Overview:

The **Smart Budget Database System** is designed to help users monitor and manage monthly household expenses by organizing purchase and return information and comparing recorded spending against predefined budgets. The system allows users to define monthly budgets for each family member, as well as separate budget limits for different expense categories such as shopping, dining, health, education, and gifts. As expenses occur—either online or in physical stores—the user records transaction details, and the system tracks spending by family member and category. When items are returned or refunds are received, the information is recorded so that net monthly spending can be accurately reflected.

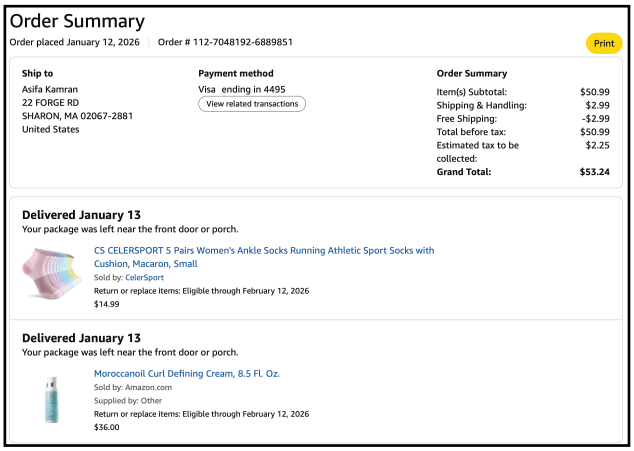
The application provides budget status updates and generates notifications when recorded spending meets or exceeds either a monthly budget or a category-specific budget. It maintains information related to users, family members, expense categories, purchases, returns, and refunds to support meaningful analysis. The system produces reports such as category-wise spending summaries, budget utilization per family member, monthly spending trends, and refund analysis to help users better understand and manage their overall budget.

All purchase and return information is entered manually by the user based on receipts and order histories.

Artifacts and Analysis

Include your analysis of at least 3 artifacts as a reference (also update it if necessary).

1.



Order Summary

Order placed January 12, 2026 Order # 112-7048192-6889851

Print

Ship to
Asifa Kamran
22 FORGE RD
SHARON, MA 02067-2881
United States

Payment method
Visa ending in 4495
View related transactions

Order Summary

Item(s) Subtotal:	$50.99
Shipping & Handling:	$2.99
Free Shipping:	-$2.99
Total before tax:	$50.99
Estimated tax to be collected:	$2.25
Grand Total:	**$53.24**

Delivered January 13
Your package was left near the front door or porch.

CS CELERSPORT 5 Pairs Women's Ankle Socks Running Athletic Sport Socks with Cushion, Macaron, Small
Sold by: CelerSport
Return or replace items: Eligible through February 12, 2026
$14.99

Delivered January 13
Your package was left near the front door or porch.

Moroccanoil Curl Defining Cream, 8.5 Fl. Oz.
Sold by: Amazon.com
Supplied by: Other
Return or replace items: Eligible through February 12, 2026
$36.00

Source of the Artifact:
It reflects a realistic order lifecycle, including itemized purchases, payment confirmation, delivery, and return eligibility. The artifact demonstrates how online shopping activity generates information that can later be recorded and analyzed for budgeting and spending management purposes.

Identified Entities

The artifact reveals several real-world entities involved in an online shopping experience, including:

- Users
- Transaction
- Purchase_item
- Product
- Category
- Item_return

Identified Actions

- Completing a transaction
- Selecting a payment method
- Receiving purchased items
- Viewing detailed order information
- Initiating a return within an allowed time period

Constraints and Business Rules

- Each transaction is uniquely identifiable.
- Transaction consist of one or more purchased items, each with its own price.

- Payment method must be completed before an order is finalized.
- Shipping charges, discounts, and taxes contribute to the final order total.
- Returns are permitted only within a defined eligibility window.
- Refund amounts cannot exceed the price originally paid for an item.
- Once an order has been delivered, it represents completed spending unless a return occurs.

2.

CVS pharmacy

66 SOUTH MAIN STREET
SHARON, MA 02067
781-784-6714
REG#13 TRN#0074 CSHR#1395960 STR#01855
January 9, 2026 03:58 PM
EXTRACARE CARD#: ********6046

3 ITEMS

F 1	RX #:****3060000	3.01N
F 1	398671 CVS ACD CNT MX 25CT	14.49T
	ORIGINAL PRICE	14.49
	BUY 1 GET 1 FOR 50% OFF	0.00-
F 1	398671 CVS ACD CNT MX 25CT	7.24T
	ORIGINAL PRICE	14.49
	BUY 1 GET 1 FOR 50% OFF	7.25-

Survey ID #
9225 2640 0158 492 10

SUBTOTAL	24.74
MA TAX 6.2500%	1.36
TOTAL	26.10

Source of the Artifact

This artifact is a point-of-sale (POS) retail receipt generated by a CVS Pharmacy store located at 66 South Main Street, Sharon, Massachusetts. It represents a completed in-store purchase and includes store identification, transaction date and time, purchased items, applied promotions, tax calculation, loyalty program usage, and final payment total. Unlike the online shopping artifact, this receipt captures real-time, in-person purchasing activity at a physical retail location.

Identified Entities

- Store
- Transaction
- Purchase_Item
- Product
- Category

Identified Actions

- Scanning items at checkout
- Applying promotional discounts (e.g., buy-one-get-one offers)
- Calculating tax based on discounted prices
- Completing payment for the transaction
- Issuing a receipt to the customer

Constraints and Business Rules

- Each transaction occurs at one store and at a specific date and time.
- A transaction consists of one or more purchase items.
- Promotional discounts apply only when predefined conditions are met.
- Discounted item prices cannot be negative.
- Taxes are calculated after promotions and discounts are applied.
- The final transaction total reflects the sum of discounted item prices plus tax.
- Once completed, an in-store transaction represents finalized spending and cannot be modified.

3.

PAYMENT BREAKDOWN	TERM	APPLIED PAYMENT AMOUNT
Target Content		
Student Services Fee-Parttime	Spring 2025	$60.00
Tuition-Graduate - Web Mining and Graph Analytics	Spring 2025	$3,900.00

Source of the Artifact

This artifact is sourced from my Boston university student account and represents institutional billing information associated with an academic term. It includes fee descriptions, term-based charges, and applied payment amounts. Unlike retail purchases, this artifact reflects structured, rule-driven billing that follows institutional policies and is typically non-refundable. It highlights a different category of financial activity that users may wish to track for overall budget awareness.

Identified Entities

- Student/Users
- Transaction
- Refund

Identified Actions

- Assessing tuition and fee charges for a specific academic term
- Generating a billing statement summarizing term-based charges
- Submitting payments toward outstanding charges
- Allocating payment amounts across multiple fees
- Updating the student's account balance after payment

Constraints and Business Rules

- Charges are associated with a specific academic term.
- Fees are determined based on enrollment status, such as part-time or full-time enrollment.
- Tuition and mandatory fees are assessed before payments are applied.
- A single payment may be applied to multiple charges.
- Applied payment amounts cannot exceed the amount of the associated charge.

- Once an academic term begins, tuition and mandatory fees are typically non-refundable.
- Posted payments and assessed charges represent finalized records and are not altered retroactively.

Use Cases

Include your use cases as a reference (also update them if necessary).

Use Case 1: Set Category Budgets

Primary Actor: User

Description:

The user defines monthly spending limits for different shopping categories such as groceries, clothing, health, gifts, and education.

Main Flow:

1. The user assigns a budget amount to each category.
2. The system records the budget information.

Outcome:

Category budgets are available for tracking and reporting purposes.

Use Case 2: Record a Purchase (Online or In-Store)

Primary Actor: User

Description:

The user records a purchase based on an online order or an in-store receipt. A purchase may be shared among multiple users, and users associated with the purchase are recorded for budgeting and tracking purposes.

Main Flow:

1. The user initiates the creation of a new Transaction.
2. The user records item-level information, including product, category, quantity, and price.
3. The user selects one or more Users to associate with the Transaction.
4. The system creates Transaction_User records to link the selected Users to the Transaction.
5. The system updates category spending totals for each associated User based on the recorded items.
6. The system validates that at least one User is associated with the Transaction.

Outcome:

The purchase is successfully recorded, associated with one or more Users, and accurately reflected in each User's budget calculations.

Use Case 3: Monitor Budgets and Generate Alerts

Primary Actor: System

Description:

The system monitors recorded spending and identifies when category budgets are reached or exceeded.

Main Flow:

1. A new purchase item is recorded.
2. The system recalculates category spending totals.
3. If spending meets or exceeds a defined budget threshold, the system generates a notification.

Outcome:
The user is informed of potential overspending and can adjust future purchasing decisions.

Use Case 4: Record Returns and Refunds
Primary Actor: User
Description:
The user records a refund and, when applicable, records a return for a previously purchased item. Refunds may be issued with or without a return.
Main Flow:

- The user selects a previously recorded **Purchase_Item**.
- The user enters **refund details** (refund amount, refund date, and refund method/status).
- The system links the **Refund** to the selected Purchase_Item.
- The system updates net spending totals for the affected **category** and **user**, reflecting the refunded amount.

Outcome:
Refunded amounts are accurately reflected in spending summaries and reports.

Use Case 5: Generate Monthly Spending and Budget Reports
Primary Actor: User
Description:
The user views reports summarizing spending behavior and budget usage for a selected month.
Main Flow:

1. The user selects a month.
2. The system aggregates recorded purchase and refund data.
3. The system calculates total spending, refunds, net spending, and remaining budget amounts.
4. The system displays category-based and summary reports.

Outcome:
The user gains clear insight into spending patterns and overall budget usage.

Summary
Collectively, these use cases describe how the system supports consistent tracking and analysis of spending across online purchases, in-store transactions, and other expense types. The focus of the system is on recording financial activity, monitoring budget usage, and providing informative feedback to support better personal budgeting decisions.

Structural Database Rules

Structural Database Rules:

1. **A User defines budgets by category.**
   A budget is set for a specific User and a specific Category.

2. **A Category can be budgeted differently by different Users.**
   The same Category (e.g., Groceries, Shopping, Pocket Money) may have different budget amounts for different Users.

3. **A Transaction records a purchase event from a store or website.**
   Each Transaction captures the purchase context (where/when the purchase occurred).

4. **A Transaction can be shared across multiple Users in the household.**
   One Transaction may include items intended for more than one User (for example, a single receipt with items for a parent and children).

5. **Each Purchase Item belongs to one Transaction and represents a specific purchased product.**
   A Transaction may include many Purchase Items.

6. **Each Product can be purchased many times over time.**
   The database stores Products so the same product can appear in many Purchase Items across different Transactions.

7. **Each Purchase Item is assigned to one spending Category.**
   Categorization supports comparing spending against category budgets.

8. A **Return is recorded for axactly one purchase item.**
   A Purchase Item may result in zero or one Return.

9. **Refunds reduce spending and are recorded for purchased items.**
   A Refund is recorded for a specific Purchase Item and decreases net spending totals.

10. **A Refund may be issued with or without a Return.**
    Some refunds are linked to returns, and some refunds are issued without requiring the item to be returned.

11. **Budget monitoring can produce Alerts when spending approaches or exceeds a budget.**
    Alerts are generated for a specific User and Category based on budget vs. net spending.

12. **Monthly Reports summarize budget performance and net spending for a User.**
    Reports are generated per User for a month and summarize spending by category, including the impact of refunds.

13. **Reports include multiple Categories and each Category can appear in many Reports over time.**
    Categories repeat across months, so reporting supports repeated inclusion.

14. **Budget limit changes are tracked over time for historical analysis.**
    When a Budget's monthlyLimit is updated, the previous monthlyLimit value is recorded in Budget_Limit_History along with the date and time of the change.

Specialization–Generalization Rules:

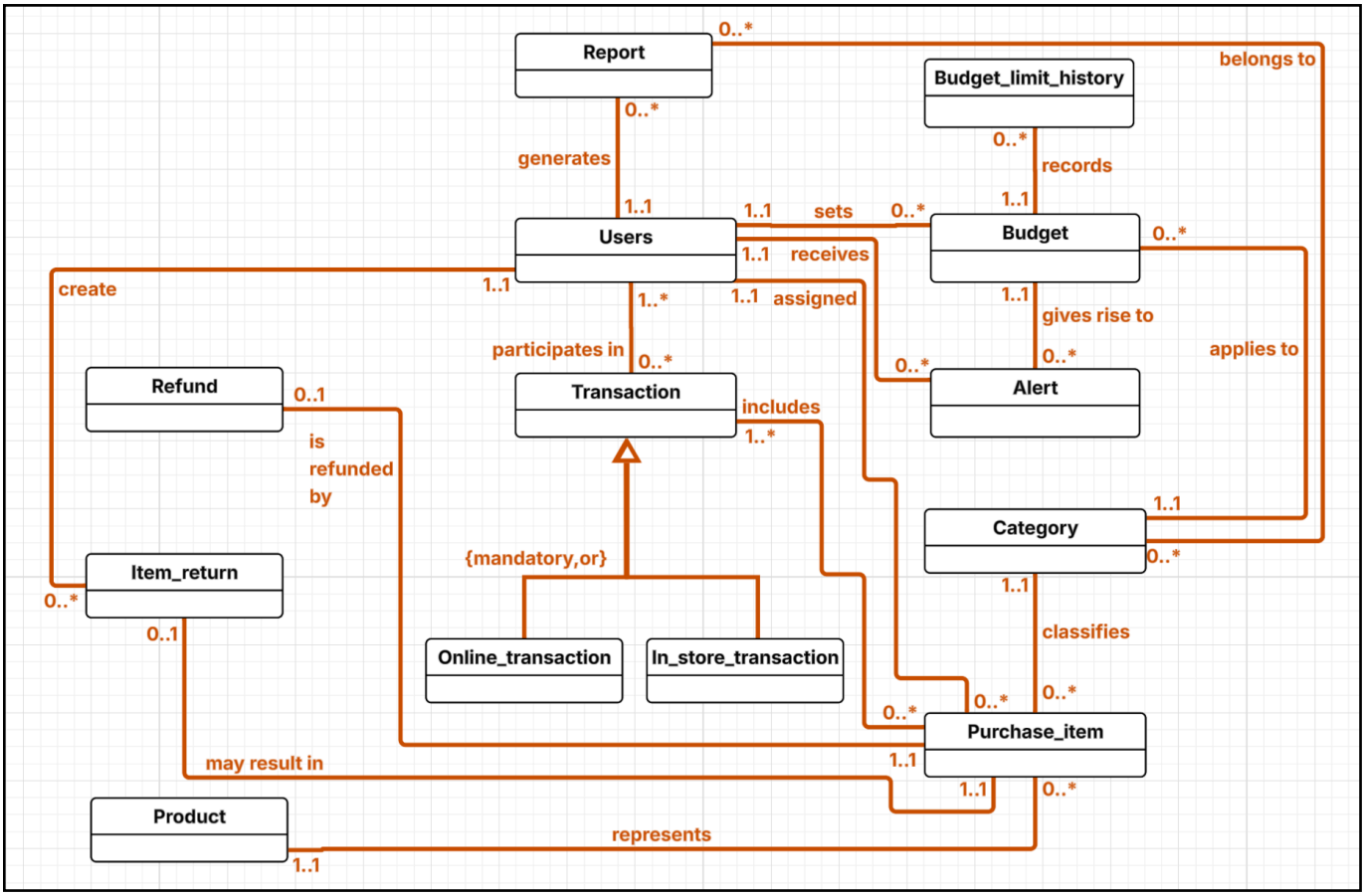
15. **A Transaction is recorded as Online or In-Store Transaction.**
    Every Transaction is either an Online Transaction or an In-Store Transaction, but not both.

Entities:

- Users
- Category
- Budget
- Transaction **(supertype):[** Online Transaction,In-Store Transaction **(subtypes)]**
- Purchase Item
- Product
- Item_return
- Refund
- Alert
- Report
- Budget_limit_history

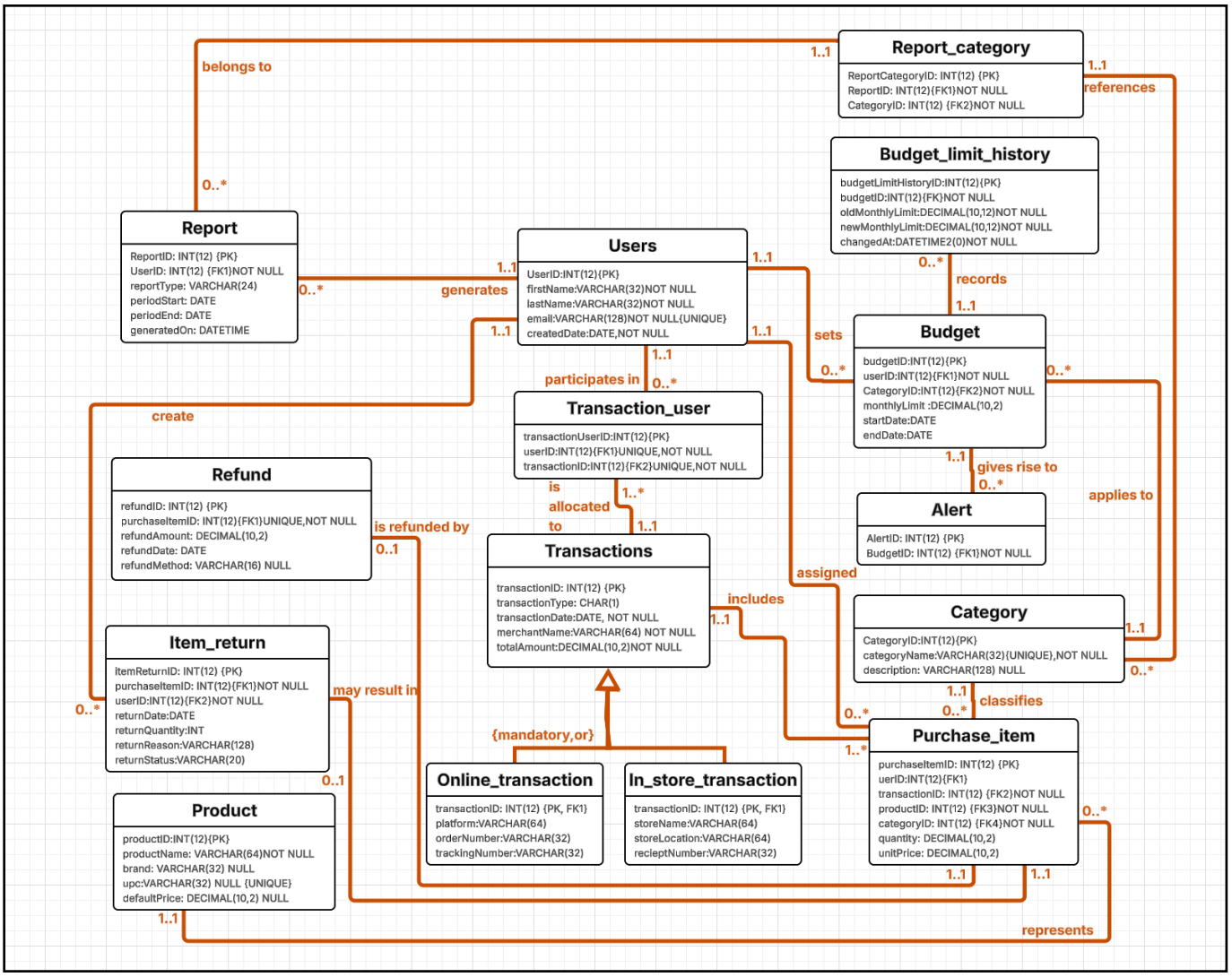
Conceptual ERD

Conceptual ERD:



Physical ERD

Physical ERD:



Attribute Reasoning

Table	Attribute	Datatype	Reasoning	Example
Users	userID(PK)	INT(12)	Unique identifier for each system user. Synthetic keys improve indexing.	1001
Users	first_name	VARCHAR(32)	Stores the user's first name for personalization in reports and alerts.	Asifa

Table	Attribute	Datatype	Reasoning	Example
Users	last_name	VARCHAR(32)	Helps distinguish users with similar names.	Kamran
Users	Email	VARCHAR(128)	Used for login and communication. Should be UNIQUE.	asifa@bu.edu
Users	CreatedDate	DATE	Tracks when the account was created for auditing and analytics.	2025-01-10

Table	Attribute	Datatype	Reasoning	Example
Budget	budgetID(PK)	INT(12)	Budgets are category-based.	10
Budget	userID(FK1)	INT(12)	Links the budget to the specific user who owns/sets the budget (enforces ownership via FK).	3
Budget	categoryID(FK2)	INT(12)	Associates the budget with one spending category (supports category-level limits and reporting).	5
Budget	monthlyLimit	DECIMAL(12,2)	Stores spending cap; DECIMAL avoids rounding errors.	400.00
Budget	startDate	DATE	Defines when the budget becomes active	2026-02-01
Budget	endDate	DATE	Allows recurring or time-bound budgets.	2026-02-28

Table	Attribute	Datatype	Explanation	Example
Transactions	transactionID(PK)	INT(12)	Unique transaction identifier.	9001

Table	Attribute	Datatype	Explanation	Example
Transactions	transactionType	CHAR(1)	Supertype discriminator ('O' Online, 'S' Store). Enforces specialization.	O
Transactions	transactionDate	DATE	Required for monthly spending reports.	2026-02-03
Transactions	merchantName	VARCHAR(64)	Helps users recognize purchases and analyze spending habits.	Amazon
Transactions	totalAmount	DECIMAL(10,2)	Needed for budget comparisons and financial summaries.	82.75

Table	Attribute	Attribute	Attribute	Attribute
Online_transaction	Transaction ID (PK, FK)	INT(12)	Ensures 1-to-1 relationship with Transaction.	900
Online_transaction	Platform	VARCHAR(64)	Identifies the websit for analytics.	Amazon
Online_transaction	orderNumber	VARCHAR(32)	Critical for tracking returns and refunds.	113-998877
Online_transaction	TrackingNumber	VARCHAR(32)	Shipping reference.	TRK839201

Table	Attribute	Datatype	Explanation	Example
In_store_transaction	Transaction ID (PK, FK)	INT(12)	Maintains subtype dependency on Transaction.	9002
In_store_transaction	StoreName	VARCHAR (64)	Allows spending analysis by retailer.	Kohls
In_store_transaction	StoreLocation	VARCHAR (64)	Useful when the same chain has multiple locations.	Walpole
In_store_transaction	ReceiptNumber	VARCHAR (32)	Required for verification and returns.	RCP102938

Table	Attribute	Datatype	Explanation	Example
Item_return	ItemReturnID (PK)	INT(12)	Unique return identifier.	880
Item_return	PurchaseItemID (FK)	INT(12)	Returns must reference the original purchase.	7001
Item_return	returnDate	DATE	Records the date the item was returned.	2026-02-15
Item_return	returnQuantity	INT	Specifies how many units were returned (supports partial returns).	1
Item_return	returnReason	VARCHAR(128)	Captures the reason for return (e.g., damaged, wrong item, defective).	"Damaged item"
Item_return	returnStatus	VARCHAR(20)	Tracks return processing status (Pending, Approved, Rejected, Completed).	Approved
Item_return	processedByUserID (FK2)	INT(12)	Identifies the user or system account that processed the return.	3

Table	Attribute	Datatype	Explanation	Example
Product	productID (PK)	INT(12)	Unique identifier for each product in the system.	300
Product	productName	VARCHAR(64)	Stores the name of the product as displayed to users.	"Organic Whole Milk"
Product	brand	VARCHAR(32)	Identifies the manufacturer or brand associated with the product.	"Horizon"
Product	Upc	VARCHAR(32)	Universal Product Code used for barcode scanning and unique product identification; stored as VARCHAR to preserve leading zeros.	"012345678905"

Table	Attribute	Datatype	Explanation	Example
Product	defaultPrice	DECIMAL(10,2)	Stores the standard price of the product; DECIMAL ensures accurate monetary representation without rounding errors.	4.99

Table	Attribute	Datatype	Explanation	Example
Report	reportID (PK)	INT(12)	Unique identifier for each generated report.	501
Report	userID(FK1)	INT(12)	References the user who generated or owns the report; enforces report ownership.	3
Report	reportType	VARCHAR(24)	Specifies the type of report (e.g., Monthly, Weekly, Yearly, Custom).	Monthly
Report	periodStart	DATE	Defines the starting date of the reporting period.	2026-02-01
Report	periodEnd	DATE	Defines the ending date of the reporting period.	2026-02-28
Report	generatedOn	DATETIME	Records the exact date and time the report was generated for audit and tracking purposes.	2026-03-01 10:45:00

Table	Attribute	Datatype	Explanation	Example
Purchase_item	PurchaseItemID (PK)	INT(12)	Unique identifier for each purchased item.	7001
Purchase_item	Transaction ID (FK)	INT(12)	Connects item to its transaction.	9001
Purchase_item	ProductID (FK)	INT(12)	Identifies what was purchased.	300
Purchase_item	CategoryID (FK)	INT(12)	Associates the item with a spending category for budget tracking and reporting purposes.	5
Purchase_item	Quantity	INT	Number of units purchased.	2
Purchase_item	UnitPrice	DECIMAL(10,2)	Accurate item pricing.	19.99

Table	Attribute	Datatype	Explanation	Example
Budget_limit_history	budgetLimitHistoryID(PK)	INT(12)	Unique identifier for each budget limit history record. Generated using a sequence to uniquely track each change event.	1001
Budget_limit_history	budgetID(FK)	INT(12)	References the Budget whose monthlyLimit was changed. Enforces that each history record belongs to an existing Budget.	45
Budget_limit_history	oldMonthlyLimit	DECIMAL(10,12)	Stores the previous monthlyLimit value before the update occurred. Preserves historical budget data for auditing and analysis.	500
Budget_limit_history	newMonthlyLimit	DECIMAL(10,12)	Records the date and time when the budget limit change occurred. Typically populated automatically using	650
Budget_limit_history	changedAt	DATETIME		

Normalization Reasoning

Normalization Approach:

The Smart Budget Database System was designed to eliminate redundancy, prevent update anomalies, and ensure data integrity. Each entity was analyzed for functional dependencies to confirm that all non-key attributes depend solely on the primary key. Repeating groups were removed, composite data was separated into distinct entities, and many-to-many relationships were resolved using associative entities with surrogate primary keys.
The database design satisfies Boyce–Codd Normal Form (BCNF).

Boyce-Codd Normal Form (BCNF) Justification:

The following validations were performed:

1. Single-Attribute Primary Key Entities:
(Users, Product, Category, Report, Budget, Transactions, Purchase_item, Refund, Alert, Budget_Limit_History, etc.)
All of these relations use a single surrogate primary key.
All non-key attributes are fully functionally dependent on that primary key.
Example:
ProductID → productName, brand, UPC, defaultPrice

No non-key attribute determines another non-key attribute.
No partial dependencies exist because the primary keys are single attributes.
Therefore, these tables satisfy BCNF.

2. Associative (Bridge) Entities:
(Transaction_user, Report_category)
Many-to-many relationships are resolved using associative entities with surrogate primary keys.
All non-key attributes (if any) depend entirely on the surrogate primary key.
No partial dependencies exist because the primary key consists of a single attribute.
No transitive dependencies exist within these tables.
Therefore, these associative entities satisfy BCNF.

3. Purchase_item Entity:
Functional dependency:
purchaseItemID → transactionID, productID, userID, categoryID, quantity, unitPrice
All attributes depend solely on the primary key.
No attribute determines another non-key attribute.
Therefore, Purchase_item satisfies BCNF.

4. Item_return and Refund Separation
Operational return details (reason, quantity, status) are separated from financial refund details (amount, method, date).
This eliminates transitive dependencies between operational and financial data and ensures that each table represents a single subject area.
Both entities independently satisfy BCNF.

5. Supertype/Subtype Structure:
(Transactions → Online_transaction / In_store_transaction)
Common attributes are stored in the Transactions supertype.
Channel-specific attributes are stored in subtype tables.
This removes null-heavy columns and prevents transitive dependencies between unrelated attributes.
Each subtype table uses the transactionID as both primary key and foreign key, preserving full functional dependency and satisfying BCNF.

Conclusion:
All relations in the Smart Budget Database System satisfy BCNF because:

- Every determinant is a candidate key.
- There are no partial dependencies.
- There are no transitive dependencies.
- Many-to-many relationships are resolved through associative entities.
- Supertype/subtype structures separate common and specialized attributes.

Therefore, the database design conforms to Boyce–Codd Normal Form.

Indexes

Include your indexes as a reference (also update it if necessary).

1) Primary Keys (PKs):

PKs are inherently UNIQUE (SQL Server enforces uniqueness).

- Users.userID
- Category.categoryID
- Product.productID
- Transactions.transactionID
- Online_transaction.transactionID (also FK to Transactions; 1:1 subtype)
- In_store_transaction.transactionID (also FK to Transactions; 1:1 subtype)
- Transaction_user.transactionUserID
- Budget.budgetID
- Budget_Limit_History.budgetLimitHistoryID
- Purchase_item.purchaseItemID
- Item_return.itemReturnID
- Refund.refundID
- Report.reportID
- Report_category.reportCategoryID
- Alert.alertID

Foreign Keys:

Column	Unique?	Description
Online_transaction.transactionID (FK → Transactions.transactionID)	Unique	This FK is also the PK of `Online_transaction` (1:1 subtype), so each online row matches exactly one transaction and cannot repeat.
In_store_transaction.transactionID (FK → Transactions.transactionID)	Unique	This FK is also the PK of `In_store_transaction` (1:1 subtype), so each in-store row matches exactly one transaction and cannot repeat.
transaction_user.userID (FK → Users.userID)	Not unique	Not unique because a user can appear in many transaction_user rows (one user can participate in many transactions).

Column	Unique?	Description
transaction_user.transactionID (FK → Transactions.transactionID)	Not unique	Not unique because a transaction can be shared by multiple users, creating multiple rows in `transaction_user`
Budget.userID (FK → Users.userID)	Not unique	Not unique because a user can define multiple budgets (across categories and/or periods).
Budget.categoryID (FK → Category.categoryID)	Not unique	Not unique because a category can be budgeted by many users (and across periods).
Alert.budgetID (FK → Budget.budgetID)	Not unique	Not unique because a budget can generate multiple alerts over time.
Report.userID (FK → Users.userID)	Not unique	Not unique because a user can have many reports (monthly, category, etc.).
Report_category.reportID (FK → Report.reportID)	Not unique	Not unique because a report can include multiple categories, so many rows can share the same
Report_category.categoryID (FK → Category.categoryID)		Not unique because a category can appear in many reports.

Column	Unique?	Description
Purchase_item.userID (FK → (Users.userID)	Not unique	Not unique because a user can have many purchase items across many transactions (one user purchases many items).

Column	Unique?	Description
Purchase_item.transaction ID (FK → Transactions.transactionID)	Not unique	Not unique because a transaction contains many purchased items.
Purchase_item.productID (FK → Product.productID)	Not unique	Not unique because the same product can be purchased many times across different
Purchase_item.categoryID (FK → Category.categoryID)	Not unique	Not unique because many purchase items can belong to the same category.
Item_return.purchaseItemI D (FK → Purchase_item.purchaseIt emID)	Not unique	Not unique because your schema does not restrict to 1 return per item (so multiple return records could reference the same purchase item).
Item_return.userID (FK → Users.userID)	Not unique	Not unique because a user can make many returns.
Refund.purchaseItemID (FK → Purchase_item.purchaseIt emID)	Not unique	Not unique because your schema does not restrict to 1 refund per item (so multiple refunds could reference the same purchase item).
Budget_limit_history.budg etID (FK → Budget.budgetID)	Not unique	Not unique because a single budget can be updated multiple times over its lifetime, resulting in multiple history records referencing the same

Query Driven Indexes:

Index Key	Unique?	Description
uq_online_order (platform, orderNumber)	Unique	Unique because an online order number should not repeat within a platform; prevents duplicate online order records.
uq_store_receipt (storeName, receiptNumber)	Unique	Unique because receipt numbers should not repeat for a given store; prevents duplicate in-store receipt records.
UX_transaction_user_userID_transactionID (userID, transactionID)	Unique	Unique because it prevents duplicate user-to-transaction assignments (same user linked to same transaction more than once).
IX_Transactions_transactionDate (transactionDate)	Not unique	Not unique because many transactions occur on the same date; supports date-range filtering.
IX_Transactions_merchantNmae (merchantName)	Not unique	Not unique because many transactions can have the same merchant; supports grouping/filtering by merchant.
IX_Budget_user_dates (userID, startDate, endDate)	Not unique	Not unique because a user can have multiple budgets across categories/periods; supports “active budgets in a month” lookups.
IX_Purchase_item_transactionID_categoryID (transactionID, categoryID)	Not unique	Not unique because many purchase items can share the same transaction and category; supports Q1/Q3 joins & NOT EXISTS logic.

Index Key	Unique?	Description
IX_transaction_user_userID (userID)	Not unique	Speeds up finding all transactions for a given user (joins/filtering in per-user summaries and reports).
IX_transaction_user_transactionID (transactionID)	Not unique	Speeds up retrieving all users tied to a transaction (splits/shared transactions) and joins back to Transactions.
IX_Budget_userID (userID)	Not unique	Speeds up retrieving all budgets for a user (budget dashboard, month budget lookups).
IX_Budget_categoryID (categoryID)	Not unique	Speeds up retrieving budgets by category across users or for category-based reporting; helps joins to Category.
IX_Alert_budgetID (budgetID)	Not unique	Speeds up finding alerts generated for a specific budget (budget monitoring/ history display).
IX_Budget_Limit_History_budgetID (budgetID)	Not unique	Speeds up retrieving budget limit changes over time for a budget (audit/ history timeline).

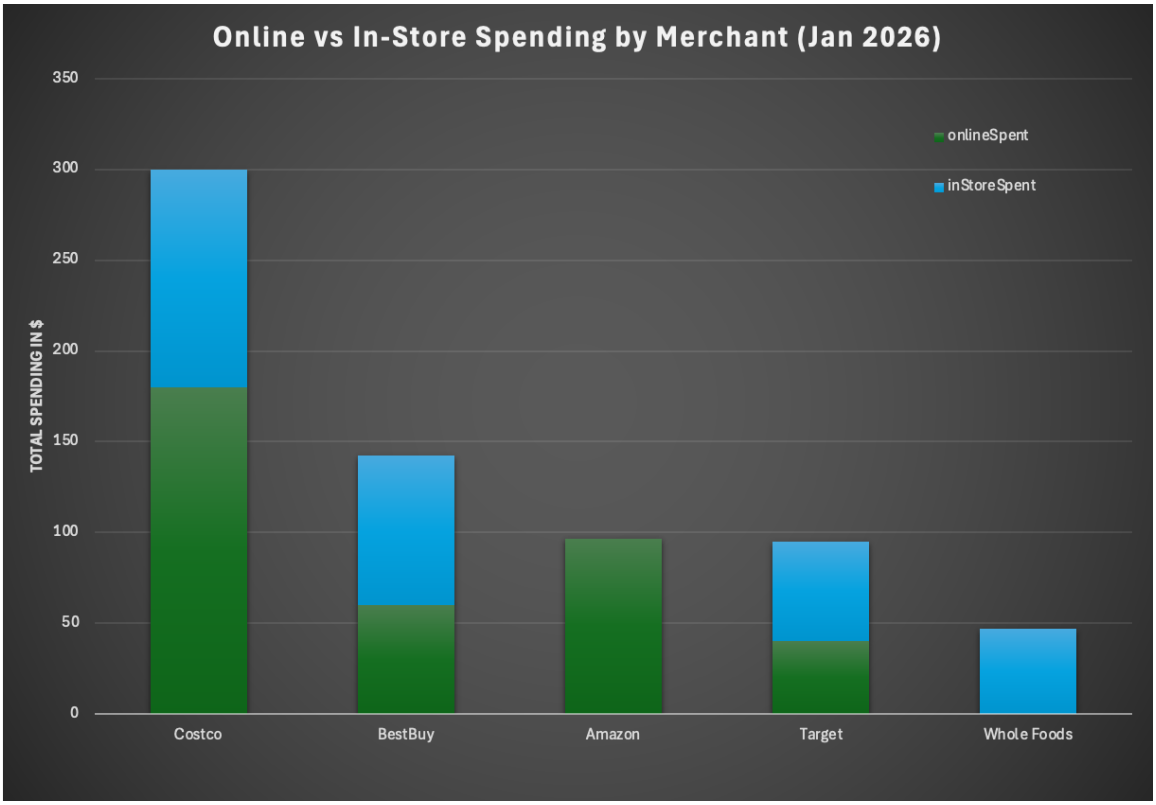
Index Key	Unique?	Description
IX_Report_category_reportID (reportID)	Not unique	Speeds up fetching all categories included in a given report (report rendering).
`IX_Report_category_categoryID (categoryID)`	Not unique	Speeds up finding all reports that include a given category (category-based report search/analytics).
IX_Purchase_item_userID (userID)	Not unique	Speeds up retrieving all items assigned to a user (per-user spending, "my purchases," user-level category totals).
IX_Purchase_item_userID (userID)	Not unique	Speeds up retrieving line items for a transaction (receipt detail view; joins from Transactions).
IX_Purchase_item_transactionID (transactionID)	Not unique	Speeds up retrieving line items for a transaction (receipt detail view; joins from Transactions).

Index Key	Unique?	Description
`IX_Purchase_item_categoryID (categoryID)`	Not unique	Speeds up category spending queries (sum by category, budget vs actual comparisons).
IX_Purchase_item_productID (productID)	Not unique	Speeds up product-based analysis (frequently purchased products, brand/product rollups) and joins to Product.
IX_Item_return_userID (userID)	Not unique	Speeds up retrieving all returns for a user (refund/return history per person).
IX_Refund_purchaseItemID (purchaseItemID)	Not unique	Speeds up locating refund records for a specific purchased item (net spend calculations, refund lookups).
IX_Transactions_transactionDate_merchantName(transactionDate, merchantName)	Not unique	Composite index to support queries filtering by date range while also grouping/filtering by merchant (e.g., "top merchants in a date range").

Data Visualizations

Create two visualizations of the data in your database. Clearly explain the data story conveyed by each visualization. Ensure that the visualizations and data stories are useful and appropriate given the intended use of your database. Add the SQL to your SQL script.

Visualization 1:



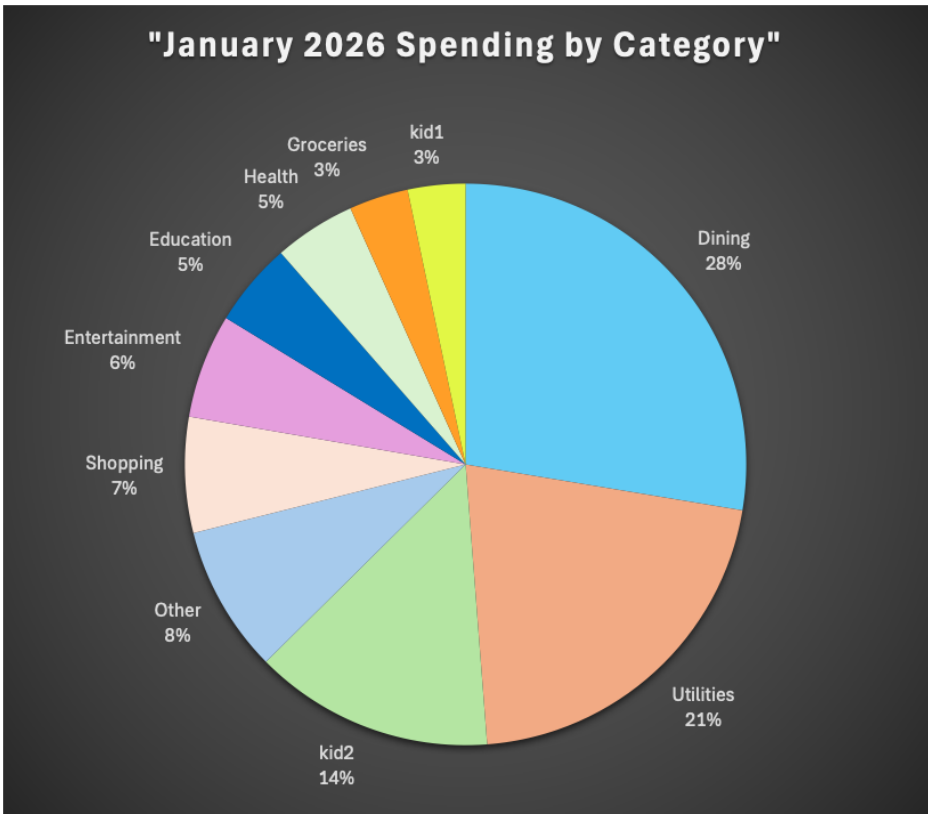
Data Story: Online vs In-Store Spending by Merchant (January 2026 – Selected User):
This chart presents the January 2026 spending patterns for the selected user in the Smart Budget Database System, highlighting total expenditure by merchant and channel preference (online vs. in-store).
Costco accounts for the highest total spending, with approximately $300 in combined purchases. A significant portion of Costco spending occurred in-store, suggesting bulk or routine household purchases made physically rather than online.
BestBuy ranks second in total spending, with a more balanced split between online and in-store purchases. This pattern may indicate electronics or higher-value items where the user compares options online before completing purchases.
Amazon spending appears entirely online, reinforcing its role as a purely digital marketplace in this dataset. This suggests convenience-driven purchases or recurring online household orders.
Target shows moderate spending distributed across both channels, indicating flexible purchasing behavior depending on product type or urgency.
Whole Foods reflects the lowest overall spending and is exclusively in-store, which aligns with typical grocery shopping behavior where fresh items are selected in person.

Overall Insight (Selected User Perspective):
For the selected user:

- Warehouse and grocery purchases (Costco, Whole Foods) skew heavily toward in-store transactions.
- Technology and general retail merchants (BestBuy, Target) show hybrid channel behavior.
- Amazon remain fully online.

This distribution highlights how channel preference for the selected user is influenced by product category, convenience, and purchasing habits within the household.

Visualization 2:



Data Story: Spending Distribution by Category (January 2026 – Selected User):

This pie chart illustrates how total spending for January 2026 for the selected user is distributed across various spending categories. The visualization highlights which categories contribute most significantly to the selected user's monthly budget.

The largest portions of spending are concentrated in Dining (28%) and Utilities (21%), indicating that recurring household costs and food-related expenses account for the majority of the selected user's expenditures during the month.

A noticeable share of spending is also allocated to kid2 (14%) and Other (8%), suggesting that household-specific and miscellaneous purchases play a meaningful role in the selected user's overall financial activity. These categories represent flexible spending areas that may fluctuate month to month depending on family needs.

Moderate spending appears in Shopping (7%) and Entertainment (6%), reflecting discretionary spending patterns that complement essential expenses. Meanwhile, smaller proportions are observed in Health (5%), Education (5%), Groceries (3%), and kid1 (3%), indicating controlled and targeted spending in these areas for the selected user.

Overall Insight (Selected User Perspective):
For the selected user, essential categories such as **Dining and Utilities** have the greatest impact on total monthly spending. Because these categories represent the largest shares, even small adjustments in these areas could significantly influence overall budget performance.
At the same time, discretionary categories remain proportionally smaller, indicating a relatively balanced spending approach. The distribution reflects a household budget that prioritizes necessities while maintaining controlled spending in lifestyle and optional categories.
Monitoring high-impact categories regularly can support more effective budget management and long-term financial stability for the selected user.